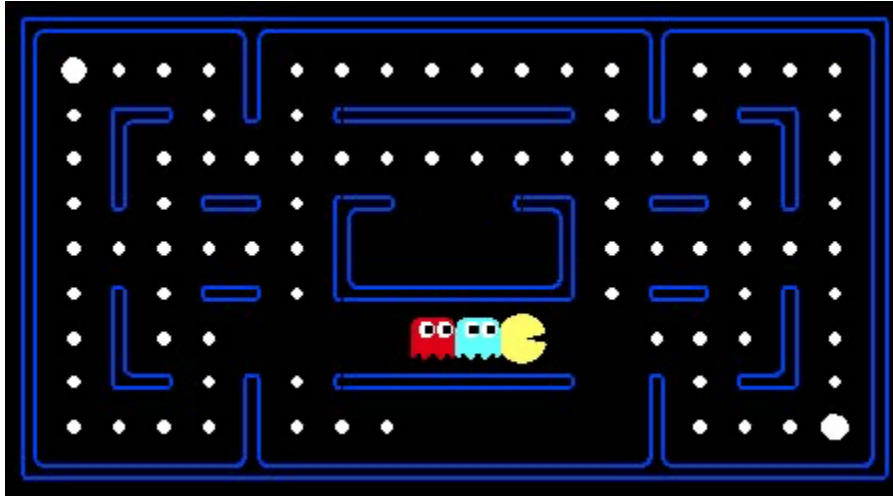


Assignment 2: Multiagent Pacman Search

150 Points



Pacman, the yellow icon in the figure above, moves around in a maze attempting to eat as many food pellets (the small white dots) as possible while avoiding the ghosts (the other characters with eyes). If Pacman consumes all the food in the maze, it wins. The large white dots at the top-left and bottom-right corners are capsules, which grant Pacman the power to eat ghosts for a limited time. However, you won't need to focus on them for the required part of the assignment. In this problem, taken from the Berkeley AI Pacman project, you will design and implement agents for the classic version of Pacman. Your agents will use minimax and expectimax searches to choose actions.

Code: The code for this problem consists of several Python files. To complete the assignment, you will need to read and understand some of these files, while others can be ignored. You can download all the necessary code and supporting files from the zip file, **assignment2.zip**, available on Canvas. Although the code base has not changed significantly from Assignment 1, please use the files included in assignment2.zip on Canvas instead of mixing files from Assignment 1.

Autograder: As in Assignment 1, this project includes an autograder for you to grade your answers on your machine. This can be run on all questions with the command:

```
> python autograder.py
```

It can be run for one particular question, such as q2, by:

```
> python autograder.py -q q2
```

It can be run for one particular test by commands of the form:

```
> python autograder.py -t test_cases/q2/0-small-tree
```

By default, the autograder displays graphics with the `-t` option, but doesn't with the `-q` option. You can force graphics by using the `--graphics` flag, or force no graphics by using the `--no-graphics` flag.

Files to Edit and Submit: You will fill in portions of `multiAgents.py` during the assignment. Please submit only this file. Do not change any other files in this distribution or submit any of our original files. Additionally, your solution should not import external packages that are unavailable in the Python Standard Library, such as `scipy`.

Evaluation: Your code will be autograded for technical correctness. Please **do not change the names of any provided functions or classes within the code**.

Collaboration Policy: Please refer to the course syllabus.

Citation Policy: Please refer to the course syllabus.

Late Policy: Please refer to the course syllabus.

Getting Help: You are not alone! If you find yourself stuck on something, contact the course staff for help on Piazza. Office hours, and Piazza are there for your support; please use them.

Discussion: Please be careful not to post spoilers on Piazza.

Code structure

File Name	Read?	Edit?	Description
multiAgents.py	Yes	Yes	Where all of your multi-agent search code resides and the only file you need to modify for this assignment.
pacman.py	Yes	No	The main file that runs Pacman games. This file describes a Pacman GameState type, which you use in this project.
game.py	Yes	No	The logic behind how the Pacman world works. This file describes several supporting types like AgentState, Agent, Direction, and Grid.
util.py	Yes	No	Useful data structures for implementing search algorithms.
graphicsDisplay.py	No	No	Graphics for Pacman.
graphicsUtils.py	No	No	Support for Pacman graphics.
textDisplay.py	No	No	ASCII graphics for Pacman.
ghostAgents.py	No	No	Agents to control ghosts.
keyboardAgents.py	No	No	Keyboard interfaces to control Pacman.
layout.py	No	No	Code for reading layout files and storing their contents.
autograder.py	No	No	Project autograder.
testParser.py	No	No	Parses autograder test and solution files.
testClasses.py	No	No	General autograding test classes.
test cases/	No	No	Directory containing the test cases for each question.
multiagentTestClasses.py	No	No	Assignment 2 specific autograding test classes.

Question 1 (15 points) - A Reflex Agent for Pacman

We have provided a reflex agent for you in multiAgents.py. You can run it with the following command line.

```
> python pacman.py -p ReflexAgent
```

The agent plays poorly, even on simple layouts.

```
> python pacman.py -p ReflexAgent -l testClassic
```

You can also try out the reflex agent on the default mediumClassic layout with one ghost or two with animation off, as follows.

```
> python pacman.py --frameTime 0 -p ReflexAgent -k 1
```

```
> python pacman.py --frameTime 0 -p ReflexAgent -k 2
```

How does your agent fare? It will likely often die with 2 ghosts on the default board, unless your evaluation function is quite good.

Options: Default ghosts are random; you can also play for fun with slightly smarter directional ghosts using the command line option `-g DirectionalGhost`. You can also play multiple games in a row with the option `-n`. Turn off graphics with option `-q` to run lots of games quickly. If the randomness is preventing you from telling whether your agent is improving, you can use `-f` to run with a fixed random seed (same random choices every game).

Now read the ReflexAgent code carefully (in multiAgents.py) and make sure you understand how it works. The reflex agent code provides some helpful examples of methods that query the GameState (a GameState specifies the full game state, including the food, capsules, agent configurations and score changes) for information. You will use these method calls in your agent code.

Now modify the evaluation function under ReflexAgent so that the agent plays better than ours. You can try adding as features, the reciprocal of important values (such as distance to food) rather than just the values themselves. Note: The evaluation function you are writing is evaluating state-action pairs; in later parts of the project, you'll be evaluating states. You may find it useful to view the internal contents of various objects for debugging. You can do this by printing the objects' string representations. For example, you can print newGhostStates with `print(newGhostStates)`.

Grading: we will run your agent on the openClassic layout 10 times. You will receive 0 points if your agent times out, or never wins. You will receive 3.75 points if your agent wins at least 5 times, or 7.5 points if your agent wins all 10 games. You will receive additional 3.75 points if your agent's average score is greater than 500, or 7.5 points if it is greater than 1000. You can try your agent out under these conditions with

```
> python autograder.py -q q1
```

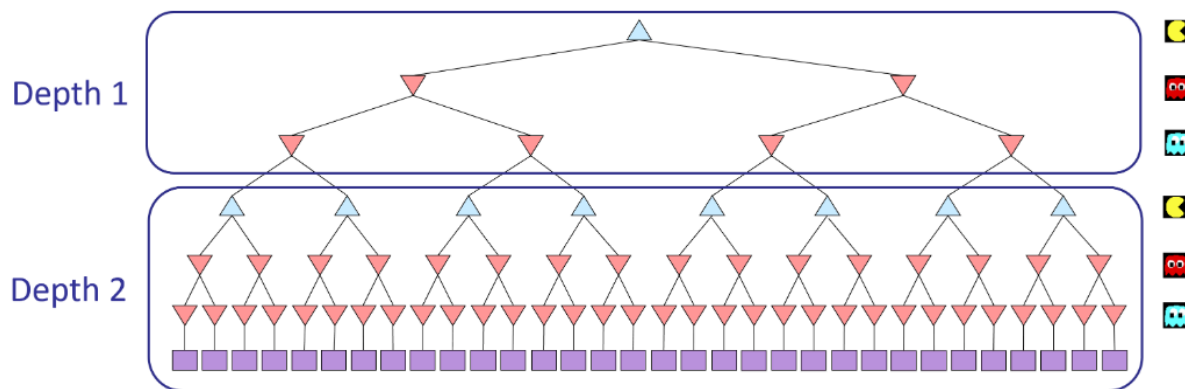
To run it without graphics, use:

```
> python autograder.py -q q1 --no-graphics
```

Question 2 (35 points) : Minimax

Pacman has multiple ghosts as adversaries. So, we will extend the minimax algorithm from class (which had only one min stage for a single adversary) to the more general case of multiple adversaries. In particular, your minimax tree will have multiple min layers (one for each ghost) for every max layer.

Important: A single search ply is considered to be one Pacman move and all the ghosts' responses, so depth 2 search will involve Pacman and each ghost moving two times (see diagram below).



You will implement depth limited minimax search with evaluation functions of the type covered in class. Suppose there are $n+1$ agents on the board, $a_0, a_1, \dots, a_n$ where a_0 is pacman and the rest are ghosts. Pacman is the max agent, and the ghosts are min agents. Write the recurrence for $V_{\text{opt}}(s, d)$, which is the minimax value with search stopping at depth d_{max} and use it to write your Python minimax search.

Now fill out the MinimaxAgent class in multiAgents.py using the above recurrence. Remember that your minimax agent should work with any number of ghosts, and your minimax tree should have multiple min layers (one for each ghost) for every max layer. Your code should also be able to expand the game tree to an arbitrary depth. Score the leaves of your minimax tree with the supplied self.evaluationFunction, which defaults to scoreEvaluationFunction. The class MinimaxAgent extends MultiAgentSearchAgent, which gives access to self.depth and self.evaluationFunction. Make sure your minimax code makes reference to these two variables where appropriate as these variables are instantiated from the command line options. Other functions that you might use in the code: GameState.getLegalActions() which returns all the possible legal moves, where each move is Directions.X for some X in the set {North, South, West, East, Stop}. Read the ReflexAgent code to see how the above are used and also for other important methods like GameState.getPacmanState(), GameState.getGhostStates() etc. These are further documented inside the MinimaxAgent class. Implement your minimaxAgent in multiAgents.py which you will upload at the end of this problem.

Hints and Observations -

- Implement the algorithm recursively using helper function(s).
- Pacman is always agent 0, and the agents move in order of increasing agent index.
- All states in minimax should be GameStates, either passed in to `getAction` or generated via `GameState.generateSuccessor`. See `GameState` in `pacman.py` for details.
- The correct implementation of minimax will lead to Pacman losing the game in some tests. This is not a problem: as it is correct behaviour, it will pass the tests.
- The evaluation function for the Pacman test in this part is already written (`self.evaluationFunction`). You shouldn't change this function but recognize that now we are evaluating states rather than actions, as we were for the reflex agent. Look-ahead agents evaluate future states whereas reflex agents evaluate actions from the current state.
- The minimax values of the initial state in the `minimaxClassic` layout are 9, 8, 7, -492 for depths 1, 2, 3 and 4 respectively. Note that your minimax agent will often win (665/1000 games for us) despite the dire prediction of depth 4 minimax.

```
> python pacman.py -p MinimaxAgent -l minimaxClassic -a depth=4
```

On larger boards such as `openClassic` and `mediumClassic` (the default), you'll find Pacman to be good at not dying, but quite bad at winning. He'll often thrash around without making progress. He might even thrash around right next to a dot without eating it because he doesn't know where he'd go after eating that dot. Don't worry if you see this behavior, question 5 will clean up all of these issues.

When Pacman believes that his death is unavoidable, he will try to end the game as soon as possible because of the constant penalty for living. Sometimes, this is the wrong thing to do with random ghosts, but minimax agents always assume the worst:

```
> python pacman.py -p MinimaxAgent -l trappedClassic -a depth=3
```

Make sure you understand why Pacman rushes the closest ghost in this case.

Grading: We will be checking your code to determine whether it explores the correct number of game states. This is the only reliable way to detect some very subtle bugs in implementations of minimax. As a result, the autograder will be very picky about how many times you call `GameState.generateSuccessor`. If you call it any more or less than necessary, the autograder will complain. To test and debug your code, run

```
> python autograder.py -q q2
```

This will show what your algorithm does on a number of small trees, as well as a pacman game.

Question 3 (30 points) : Alpha-Beta Pruning

Make an agent that uses alpha-beta pruning to more efficiently explore the minimax tree, in AlphaBetaAgent in multiAgents.py. Again, your algorithm will be slightly more general than the pseudo-code in the slides, so part of the challenge is to extend the alpha-beta pruning logic appropriately to multiple minimizer agents. You should see a speed-up (depth 3 alpha-beta will run as fast as depth 2 minimax). Ideally, depth 3 on smallClassic should run in just a few seconds per move or faster.

```
> python pacman.py -p AlphaBetaAgent -a depth=3 -l smallClassic
```

The AlphaBetaAgent minimax values should be identical to the MinimaxAgent minimax values, although the actions it selects can vary because of different tie-breaking behavior. Again, the minimax values of the initial state in the minimaxClassic layout are 9, 8, 7 and -492 for depths 1, 2, 3 and 4 respectively. Make the changes in multiAgents.py which you will upload at the very end.

Grading: Because we check your code to determine whether it explores the correct number of states, it is important that you perform alpha-beta pruning without reordering children. In other words, successor states should always be processed in the order returned by GameState.getLegalActions. Again, do not call GameState.generateSuccessor more than necessary.

You must not prune on equality in order to match the set of states explored by our autograder. (Indeed, alternatively, but incompatible with our autograder, would be to also allow for pruning on equality and invoke alpha-beta once on each child of the root node, but this will not match the autograder.)

The pseudo-code below represents the algorithm you should implement for this question.

Alpha-Beta Implementation

α : MAX's best option on path to root
 β : MIN's best option on path to root

```
def max-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = -\infty$   
    for each successor of state:  
         $v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v > \beta$  return  $v$   
         $\alpha = \max(\alpha, v)$   
    return  $v$ 
```

```
def min-value(state,  $\alpha$ ,  $\beta$ ):  
    initialize  $v = +\infty$   
    for each successor of state:  
         $v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$   
        if  $v < \alpha$  return  $v$   
         $\beta = \min(\beta, v)$   
    return  $v$ 
```


To test and debug your code, run:

```
> python autograder.py -q q3
```

This will show what your algorithm does on a number of small trees, as well as a pacman game.

To run it without graphics, use:

```
> python autograder.py -q q3 --no-graphics
```

The correct implementation of alpha-beta pruning will lead to Pacman losing some of the tests. This is not a problem: as it is correct behavior, it will pass the tests.

Question 4 (40 points) : Expectimax

Minimax and alpha-beta are great, but they both assume that you are playing against an adversary who makes optimal decisions. As anyone who has ever won tic-tac-toe can tell you, this is not always the case. In this question you will implement the ExpectimaxAgent, which is useful for modeling probabilistic behavior of agents who may make suboptimal choices.

- (30 points) As with the search problems covered so far in this class, the beauty of the algorithms is their general applicability. To expedite your own development of the Expectimax Agent in multiAgents.py, we've supplied some test cases based on generic trees. You can debug your implementation on small game trees using the command:

```
> python autograder.py -q q4
```

Debugging on these small and manageable test cases is recommended and will help you to find bugs quickly. Once your algorithm is working on small trees, you can observe its success in Pacman. ExpectimaxAgent, will no longer take the minimum over all ghost actions, but the expectation according to your agent's model of how the ghosts act. To simplify your code, assume you will only be running against an adversary which chooses amongst their getLegalActions uniformly at random. You will upload multiAgents.py at the very end of the exercise.

- (10 points) Now your ExpectimaxAgent debugged in the last part, will have your agent no longer taking the min over all ghost actions, but the expectation according to your agent's model of how the ghosts act. Assume pacman is playing against RandomGhosts, each of which chooses from its legal actions (getLegalActions) uniformly at random. To see how the ExpectimaxAgent behaves in Pacman, run:

```
> python pacman.py -p ExpectimaxAgent -l minimaxClassic -a depth=3
```

You should now observe a more cavalier approach in close quarters with ghosts. In particular, if Pacman perceives that he could be trapped but might escape to grab a few more pieces of food, he'll at least try. Investigate the results of these two scenarios:

```
> python pacman.py -p AlphaBetaAgent -l trappedClassic -a depth=3 -q -n 10
```

```
> python pacman.py -p ExpectimaxAgent -l trappedClassic -a depth=3 -q -n 10
```

This question is autograded using these two commands. You should find that your ExpectimaxAgent wins about half the time, while your AlphaBetaAgent always loses. Make sure you understand why the behavior here differs from the minimax case. The correct implementation of expectimax will lead to Pacman losing some of the tests. This is not a problem: as it is correct behavior, it will pass the tests.

Question 5 (30 points) : Evaluation Function

Write a better evaluation function for pacman in the provided function `betterEvaluationFunction`. The evaluation function should evaluate states. You may use any tools at your disposal for evaluation, including uniform cost and A* search from the previous assignment. With depth 2 search, your evaluation function should clear the `smallClassic` layout with one random ghost more than half the time and still run at a reasonable rate (to get full credit, Pacman should be averaging around 1000 points when he's winning).

```
> python autograder.py -q q5
```

```
> python pacman.py -l smallClassic -p ExpectimaxAgent -a evalFn=better -q -n 10
```

Grading: the autograder will run your agent on the `smallClassic` layout 10 times. We will assign points to your evaluation function in the following way:

- If you win at least once without timing out the autograder, you receive 5 points. Any agent not satisfying these criteria will receive 0 points.
- +5 for winning at least 5 times, +10 for winning all 10 times
- +5 for an average score of at least 500, +10 for an average score of at least 1000 (including scores on lost games)
- +5 if your games take on average less than 30 seconds on the autograder machine.
- The additional points for average score and computation time will only be awarded if you win at least 5 times.

Hints: You may want to use the reciprocal of important values (such as distance to food) rather than the values themselves. One way you might want to write your evaluation function is to use a linear combination of features. That is, compute values for features about the state that you think are important, and then combine those features by multiplying them by different values (weights) and adding the results together. You might decide what to multiply each feature by based on how important you think it is.